

# Task 5: Short Answer Questions

Ajibola Akelebe · Frontend Developer, Branda V2 · October 2026

---

## 1. A frontend project with significant traffic, complex functionality or a large user base. What was your role?

At Bema Integrated Services, I'm a Full-Stack Software Engineer (frontend-strong), and I lead the customer-facing features and integrations on BemaHub. The work I own includes a sales onboarding wizard, attribution links for growth campaigns, checkout recovery for Paystack and PayPal, and consent-aware analytics with GTM and GA4. It's complex because money and tracking meet in the browser. A payment can succeed after someone has closed the tab, so every return path has to settle correctly. We keep it safe with more than 850 Vitest tests in CI and Playwright journeys on mobile and desktop.

Before Bema, I built the Zora Market app with Expo: one codebase for iOS and Android, now live on the App Store and Google Play. The same codebase also serves a marketing site, a vendor dashboard and an admin dashboard.

## 2. Which frontend technologies are you strongest in, and why? Include your Next.js experience.

Next.js, React, TypeScript and Tailwind CSS. I've used Next.js from version 13 to 16 on client work and on my own products: Narvo News, the Zora web surfaces, Nego and my portfolio. I like that it lets me decide page by page what runs on the server and what runs in the browser, and that decision is most of performance. For mobile I use React Native with Expo.

I've spent about a decade working across design and engineering. That's why I care about the gap between the Figma file and the browser: spacing, type, motion, and how the page feels on a cheap phone.

## 3. When would you choose SSR, SSG, ISR or client-side rendering?

- **SSG** for pages that are the same for everyone: service pages, the about page. Fastest, and cheapest to serve.
- **ISR** for content that changes sometimes, like prices or a catalogue fed by a CMS. It stays fast and refreshes when the data changes.
- **SSR** when the page depends on the request: search results from URL filters, or an order confirmation.
- **Client-side** for private, interactive state that search engines don't need: the cart, a form in progress, a dashboard widget.

Branda's build uses static, per-request and client rendering, and the README shows which page uses which. ISR comes in once prices move to a CMS.

## 4. How do you decide between Server and Client Components?

Server by default. A component becomes a Client Component only when it needs the browser: clicks, state, effects, `localStorage`. Then I make that client part as small as possible. In Branda the product card is a Server Component, and only the "+" button inside it is a Client Component. The card's markup ships as HTML, not JavaScript.

## 5. How do you identify and fix slow performance?

I measure before I touch anything. Lighthouse on a throttled mobile profile, the Performance and Network tabs in Chrome, the React Profiler, and real-user data where it exists. The cause is usually one of three things: images that are too big, too much JavaScript, or third-party scripts. I fix the biggest one, then measure again to prove it helped. Building Heka IQ for Nigerian users on 3G taught me to test on a slow connection, not my laptop's.

## 6. Your approach to reusable, scalable React components?

I don't make something reusable until I need it in a third place. When I do, I keep the props small, use native HTML elements so accessibility comes free, and keep shared logic in one function, not copied around. In Branda, one quantity stepper serves the cart drawer, cart page and product page, and one `orderSummary()` calculates every total. That second one fixed a real bug: the old cart page showed a delivery fee but never added it to the total.

## 7. How do you handle loading, error and empty states in Next.js?

- **Loading:** `loading.tsx` or `Suspense` with skeletons the same shape as the real content, so nothing jumps when it arrives.
- **Errors:** `error.tsx` per section with a "Try again" button, and the error reported to Sentry or GlitchTip so I see it before users complain.
- **Empty:** designed like any other screen, with a clear next step. In Branda, a search with no results says "Nothing matches that" and offers to show all services.

## 8. How would you handle SEO for a multi-market site?

Subfolders (`/ng`, `/us`, `/uk`, `/ca`) so every market builds up the same domain. Each page gets a canonical URL, `hreflang` links to its siblings in the other markets plus an `x-default`, and its own title and description. The sitemap lists every market version. Service pages include product structured data with the local price and currency. And the content itself should be local: different headlines and featured services per market, not just a currency swap. All of that is in this build.

## 9. How do you make sure a site works across screen sizes and browsers?

I design and build mobile first, starting at about 390px. At Bema, Playwright runs our key journeys at mobile and desktop sizes on every change. I also test on real devices; shipping Zora through both app stores taught me how much an iPhone and a mid-range Android can differ. Then keyboard-only navigation and reduced-motion settings, because those break quietly.

## 10. What tools do you use for debugging?

Chrome DevTools first: Elements, Network, Performance. Then React DevTools, the Next.js error overlay, Safari's Web Inspector for iPhone-only bugs, and Playwright's trace viewer when a test fails in CI. In production, Sentry or GlitchTip. I also use AI coding assistants like Claude and Codex to read unfamiliar code faster, but I check what they tell me before I act on it.

## 11. What steps do you take to improve Core Web Vitals?

- **LCP:** preload the hero image at the right size, load fonts with `next/font`, serve static HTML from the CDN where possible.

- **INP:** ship less JavaScript, keep heavy work out of click handlers, and load third-party scripts after the page is interactive.
- **CLS:** reserve space for every image and embed, and make loading skeletons match the final layout.

Then I watch real-user numbers after release, not just the lab score.

## 12. A difficult frontend bug, and how you solved it

On the Zora Market app (React Native and Expo), several screens showed only the header and an empty body on a real phone: the password reset screens, change password, and the first step of onboarding. The phone verification overlay was worse. It was invisible, but it still blocked every tap. Nothing had crashed. The screens were there, just invisible.

The cause was an entrance animation. Each screen started its content at opacity 0 and relied on a native-driven fade to reveal it. On device that animation didn't run, so the content never appeared.

The hard part was that I'd already fixed this. In February I changed the same pattern across about two dozen screens. Then a May rework of onboarding brought it back, and the newer password screens copied the old pattern. A fix that depends on everyone remembering a rule doesn't last.

So I fixed it twice over. Every screen now starts at its final, visible state, and the animation is only decoration. Then I added a test that scans the app and fails CI if any fade or opacity value starts at 0, with a short allow-list for purely decorative splash effects. It can't quietly come back a third time.

What I took from it: animation should decorate content that's already visible, never be what reveals it. And when a bug returns, the second fix should be a test.

## 13. How do you keep code maintainable on a large team?

Agree on a few rules and let the tools enforce them: strict TypeScript, lint and tests in CI, and no merge until they pass. Keep pull requests small enough to review properly. Put shared logic in one place so two parts of the app can't disagree. Write tests where mistakes cost money. At Bema, every ticket separates what was observed from what is only assumed, which stops a lot of wasted effort. Teaching helps too. I teach software engineering on the Trinity University Skills Programme and at Lagos Data School, and if I can't explain a piece of code to a student, it's usually too clever for a teammate as well.